

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CASE STUDY

Course Code:	ITA03	Course Title:	Mobile computing
CO Assessed:	CO4	Assessment:	AT2 – CASE STUDY
Weightage:	30 % of CIA	Total Marks:	100
CO4 Statement:	<i>Identify the components and structure of mobile application for Android and Windows OS-based devices.</i>		
Student Name:	Rose Mystica. J	Reg. No.:	192421410
Section / Batch:	D / 3 rd Year	Date:	18.826

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Duration: 60 minutes
- Number of questions : 01

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Annexure A - QUESTIONS

34. A company notices that an application performs differently on Android devices from different manufacturers even though all devices use Android. Some devices provide faster execution and better battery life than others. Analyze how differences in hardware abstraction, kernel configuration, runtime implementation, and manufacturer customization could explain the observed behavior.

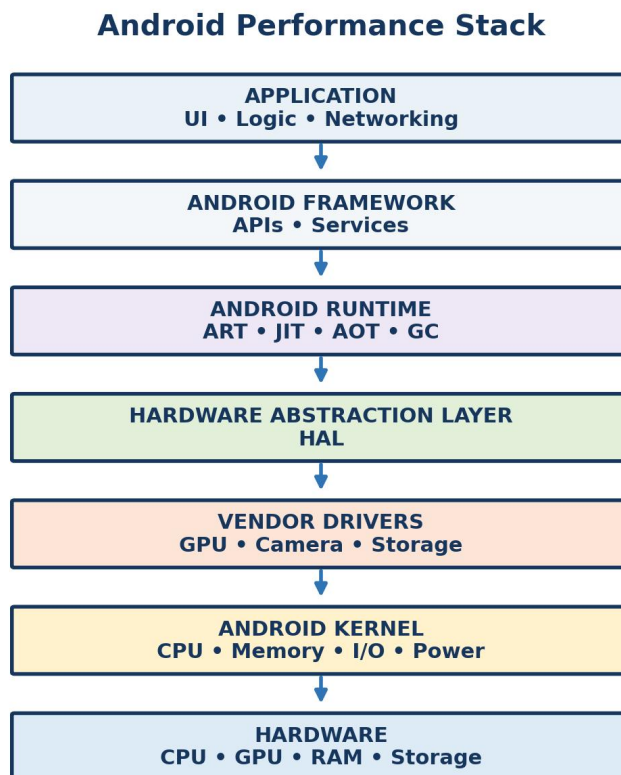
1. INTRODUCTION:

Android provides a common application platform, but devices from different manufacturers do not contain identical hardware or identical low-level software. The same application therefore passes through different drivers, kernel policies, runtime conditions and manufacturer-specific services.

Performance differences may appear as faster execution, smoother graphics, shorter startup time, lower battery drain, or better sustained performance. The main reasons can be organized into four layers: hardware abstraction, kernel configuration, Android Runtime (ART), and manufacturer customization.

2. ANDROID PERFORMANCE ARCHITECTURE:

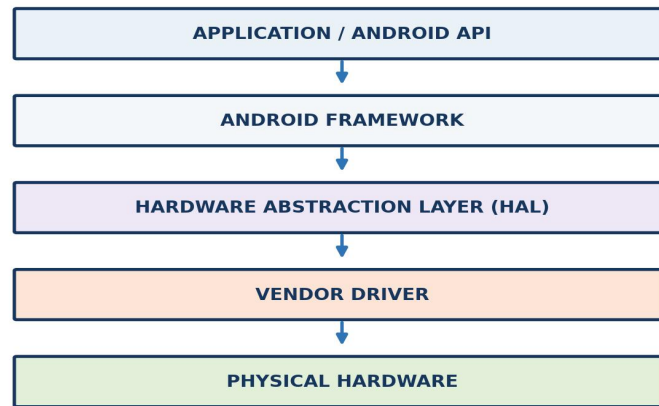
An Android application does not communicate directly with the physical processor, memory or graphics hardware. Requests pass through several layers before reaching the hardware.



3. HARDWARE ABSTRACTION & HARDWARE DIFFERENCES

The Hardware Abstraction Layer (HAL) provides a standard interface between Android framework components and hardware-specific implementations. This allows applications to use common Android APIs while manufacturers supply implementations for their particular hardware.

Hardware Access Flow



Hardware factors that affect application performance:

Component	Possible difference	Effect on application
CPU	Core architecture, frequency and core types	Different computation speed and scheduling options
GPU	Architecture and vendor driver	Different rendering speed and frame stability
RAM	Capacity and bandwidth	Different multitasking and memory pressure
Storage	UFS/eMMC generation and controller	Different app startup and database I/O
Display	Resolution and refresh rate	Different graphics workload
Modem / sensors	Chipset and firmware	Different background power and response behavior

Example – Graphics workload:

Two devices can expose the same Android graphics API while using different GPUs and vendor drivers. One device may sustain a higher frame rate, while another may show higher frame-time variation or lower sustained performance.

DEVICE A Optimized driver + stronger GPU	DEVICE B Different driver + lower GPU capability
--	--

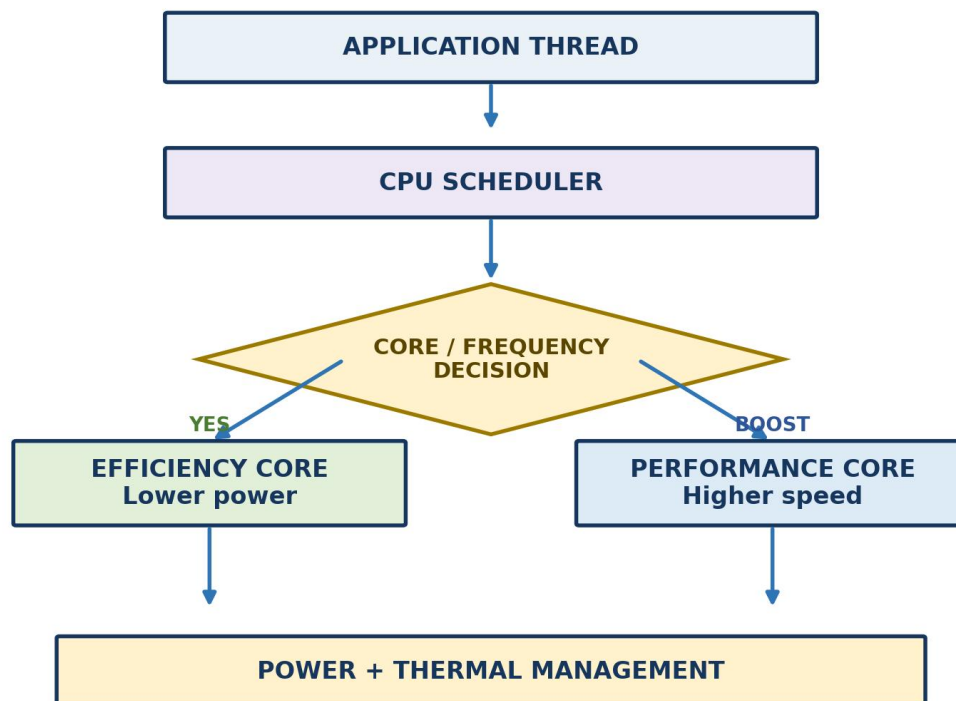
Key observation:

Hardware abstraction provides compatibility, not identical performance. The same application call can eventually reach different hardware paths

4. KERNEL CONFIGURATION & RESOURCE MANAGEMENT

The Android kernel controls important system resources including CPU scheduling, memory management, storage I/O, power states and thermal behavior. Manufacturers can tune these policies according to their performance, battery and temperature targets.

Kernel Scheduling and Power Decision



Kernel areas that can differ:

Kernel area	Manufacturer-level difference	Observed result
CPU scheduler	Task placement and load balancing	Different responsiveness
DVFS	Frequency and voltage thresholds	Different burst speed and power
Thermal management	Throttling thresholds and curves	Different sustained performance
Memory management	Reclaim, compression and process limits	Different multitasking behavior
I/O scheduling	Storage request handling	Different I/O latency
Power management	Sleep and wake-up policy	Different battery consumption

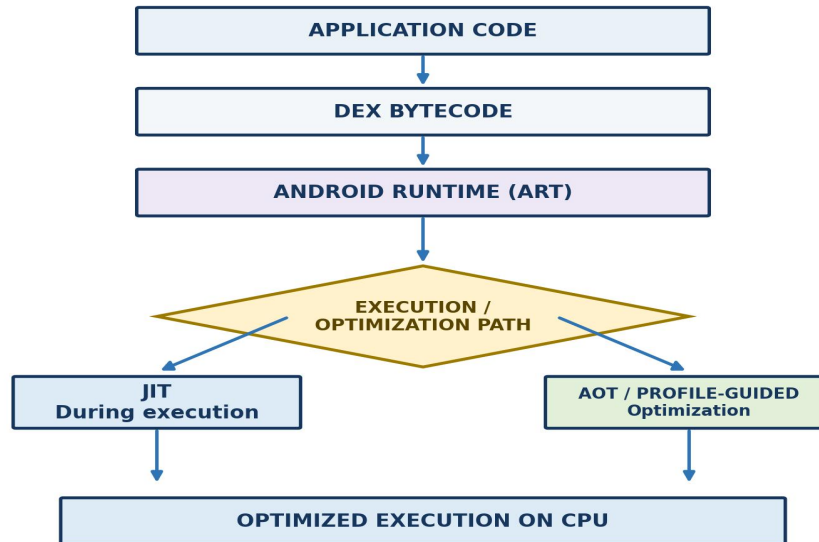
Thermal throttling:

A device can initially perform very well but slow down during a long workload. High CPU or GPU utilization raises temperature; once a thermal limit is reached, the system can reduce frequency to control heat. This creates different sustained performance even when peak specifications look similar.

5. ANDROID RUNTIME (ART) & EXECUTION

Android applications are distributed using DEX bytecode and are executed through Android Runtime (ART). ART can use JIT compilation, ahead-of-time compilation and profile-guided optimization. Runtime behavior also depends on CPU speed, memory availability and background system activity.

ART Execution and Optimization Flow



Runtime mechanisms and their effects:

Mechanism	Function	Possible performance effect
JIT compilation	Compiles frequently executed code during execution	Warm-up cost followed by faster hot paths
AOT / profile-guided optimization	Prepares or optimizes frequently used code	Can improve startup or repeated execution
Garbage collection	Reclaims unused objects	CPU work and possible pauses
Thread scheduling	Coordinates application and runtime threads	Different responsiveness under load
Heap / memory limits	Controls allocation and memory pressure	Different GC activity or app reloads

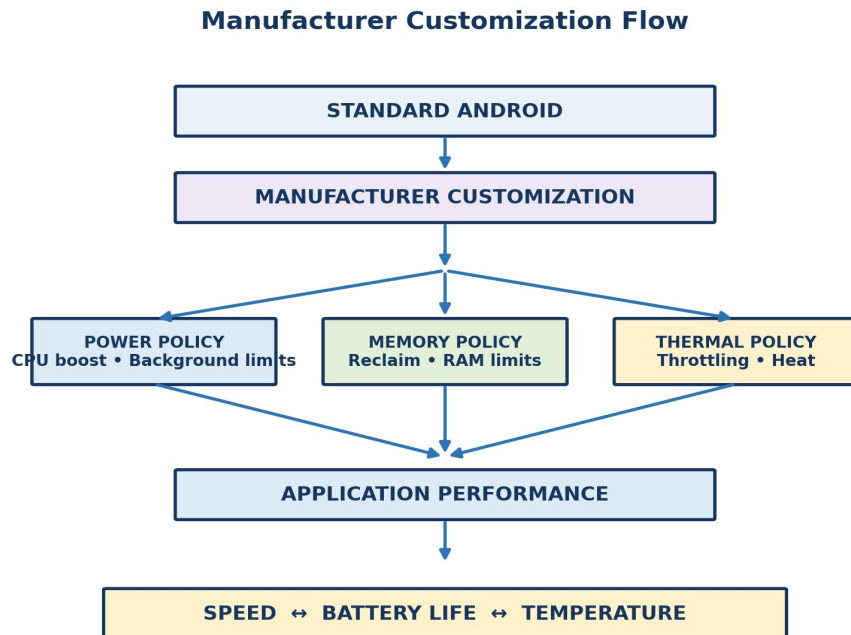
Why the same APK can behave differently:

The APK may be identical while the surrounding execution conditions differ. Device CPU capability, ART optimization state, memory pressure, background services and OEM policies can all change how much CPU time and memory are available to the application.

Same APK	Device A	Device B
Application package	Faster CPU / lower background load	Different CPU / higher background load
Runtime state	Different profile / cache state	Different profile / cache state
Result	Faster execution	Slower or more variable execution

6. MANUFACTURER CUSTOMIZATION

Manufacturers commonly add their own power, memory, thermal, graphics, networking and background-process policies. These customizations can improve battery life or device responsiveness, but they can also cause the same application to behave differently across devices.



Common manufacturer customizations:

Customization	Typical mechanism	Application effect	Battery effect
Power manager	CPU boost and background limits	May delay or accelerate work	Often reduces drain
Memory manager	Process reclaim / RAM optimization	May cause app reloads	Can reduce background use
Thermal manager	Vendor throttling curves	Changes sustained performance	Controls heat
Graphics stack	Vendor compositor / GPU libraries	Changes FPS and frame latency	GPU power varies
Networking	Sync and radio policies	Changes background timing	Radio drain varies
Preinstalled services	Security / cloud / analytics	Consumes CPU, RAM or network	Adds background use

Performance versus battery trade-off:

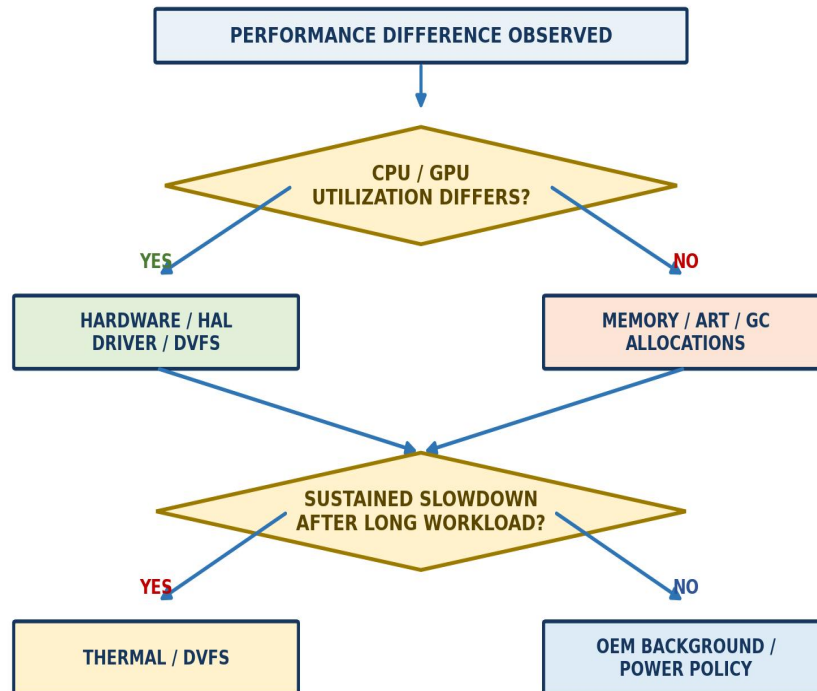
Performance-oriented policy	Battery-oriented policy
Higher CPU/GPU boost	Lower sustained frequency
Faster response	Lower power consumption
More heat	Lower temperature
Potentially shorter battery life	Potentially longer battery life

Therefore, better battery life does not necessarily mean weaker hardware. It can indicate a more conservative scheduler, power policy, background restriction or thermal strategy.

7. INTEGRATED ANALYSIS & ROOT-CAUSE IDENTIFICATION

When the company observes different application performance, the most useful approach is to identify which layer is producing the difference rather than assuming that the application code is the only cause.

Layer-by-Layer Root-Cause Analysis



Performance measurement matrix:

Metric	Purpose	What it can reveal
Startup time	Compare cold and warm launch	Storage, ART or initialization
CPU utilization / frequency	Observe scheduling and scaling	Kernel and DVFS behavior
GPU utilization / FPS	Measure graphics path	GPU and driver differences
Memory / GC pauses	Measure runtime pressure	ART, allocations or OEM memory policy
Temperature	Detect throttling	Thermal management
Battery drain	Compare energy efficiency	Power and background behavior
Network activity	Track background communication	Radio and synchronization policy

8. Final Conclusion:

For the company, the appropriate solution is not to assume that all Android devices will behave identically. Instead, performance should be validated using a device matrix and layer-by-layer profiling.

FINAL TAKEAWAY → Android provides a common application platform, but performance must be validated on real devices because lower-level implementations and manufacturer policies differ.